20 | 08 | 2026

Diseño de Compiladores

Describir un lenguaje de listas de números enteros positivos separados por comas ";" (lista unitaria se admite)

lista → lista , nro | nro { lista → nro R ; R → , nro R | ε

ó

lista → nro , lista | nro { lista → nro R ; R → , lista | ε

nro → nro dig | dig { nro → dig R' ; R' → dig R' | ε

dig → 0 | 1 | ... | 9

BNF

lista → nro R
R → , nro R | ε
nro → dig R'
R' → dig R' | ε
dig → 0 | 1 | ... | 9

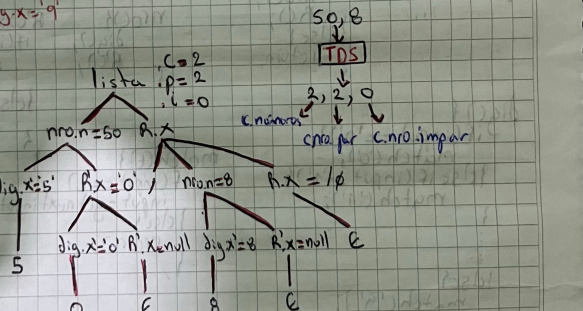
① R.x I. : No ② F.C x I : No

③ F.C. I x I : PRIM

P(lista) = {0,1,...,9}
P(R) = {,} ∪ {ε} = {,, ε}
P(nro) = {0,1,...,9}
P(R') = {0,1,...,9} ∪ {ε} = {0,1,...,9, ε}
P(dig) = {0} ∪ {1} ∪ ... ∪ {9} = {0,1,...,9}

```
lista(){
    nro();
    R();
}

R(){
    if(input==','){
        match(',');
        nro();
        R();
    }else
        return;
}

nro(){
    dig();
    R'();
}

R'(){
    if(input>='0' && input<='9'){
        dig();
        R'();
    }else
        return;
}

dig(){
    if(input=='0')
        match('0');
    else if(input=='1')
        match('1');
    .
    .
    .
    else
        match('9');
}

match(t){
    if(input==t)
        input=getnextinput();
    else
        error();
}
```

BNF	Reglas Semánticas
lista → nroR	if(cator(nro.n) % 2 == 0) lista.p = 1 + R.p else lista.i = 1 + R.i ; ~~p = if(listvar, lista.i);~~
R → ,nroR	R.c = 1 + R.c , if(cator(nro.n) % 2 == 0) R.p = 1 + R.p else R.i = 1 + R.i;
R → ϵ	R.c = 0 ; R.i = 0 ; R.p = 0
nro → digR'	nro.n = dig.x ‖ R'.x
R' → digR'	R'.x = dig.x ‖ R'.x
R' → ϵ	R'.x = '\0'
dig → 0	dig.x = '0'
dig → 1	dig.x = '1'
...	;
dig → 9	dig.x = '9'




```
lista(){
  int c,p,i;
  if(cator(nro.n)%2==0){
      p=1, c=1, i=0
  }else
      i=1; c=1; p=0

  c=c+R(&i,&p)   c=c+R.c
                 i=i+R.c
                 p=p+R.p
}
```

```
dig(){
  if(input =='0'){
      match('0');
      return('0');
  }else if(input =='1'){
      match('1');
      return('1');
  }
  ⋮
  }else{
      match('9');return('9');
  }
}
```

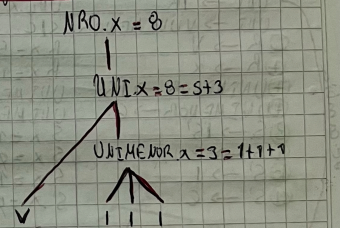
```
R(){
  int c,p,i;
  if(input == ','){
      match(',');
      nro();
      R();
  }else{
      c=0; i=0; p=0;
      return(c,i,p);
  }
}
```

```
R'C){
    if (input == '0' || input == '1' || ... input == '9'){
        return (dig || R'C');
    }else
        return ('10');
    }

nro C){
    return (dig || R'C');
}
```

Escribir un TDS que acepte un lenguaje formado por números romanos del 1 al 10 y retorne el número correspondiente

Ejemplo:

VIII → TDS → 8



BNF propuesto:

NRO → DEC | UNI

DEC → X | ϵ

UNI → UNIMENOR | IV | VUNIMENOR | IX

UNIMENOR → I | II | III | ϵ

① DIRIGIDE

BNF	Reglas Semánticas
① NRO → DEC	NRO.x = DEC.x
② NRO → UNI	NRO.x = UNI.x
③ DEC → X	DEC.x = 10
DEC → ϵ	DEC.x = 0
UNI → UNIMENOR	UNI.x = UNIMENOR.x
UNI → IV	UNI.x = 4
UNI → VUNIMENOR	UNI.x = 5 + UNIMENOR.x
UNI → IX	UNI.x = 9
UNIMENOR → I	UNIMENOR.x = 1
UNIMENOR → II	UNIMENOR.x = 2
UNIMENOR → III	UNIMENOR.x = 3
UNIMENOR → ϵ	UNIMENOR.x = 0

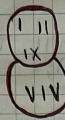
(2) PREDICTIVA

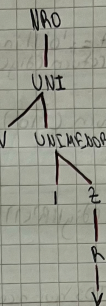
- $R_{x,i+q} = NO$
- F.C.dir & ind

Resolviendo F.C.dir

$$\begin{cases} UNI \rightarrow UNIMENOR \mid IR \mid VUNIMENOR \\ R \rightarrow VIX \end{cases}$$

$$\begin{cases} UNIMENOR \rightarrow IS \mid \epsilon \\ S \rightarrow IS' \mid \epsilon \\ S' \rightarrow II\epsilon \end{cases}$$





		Sin cambiar
	UNI → UNIMENOR	S/C
	UNI → IR	UNI.x = 1+R.x
	UNI → VUNIMENOR	S/C
(2)	R → V I	R.x = 5 3
(3)	R → X	R.x = 10 8
(4)	UNIMENOR → IS	UNIMENOR.x = 1+S.x
(5)	UNIMENOR → ϵ	UNIMENOR.x = ∅
(6)	S → IS'	S.x = 1+S'.x
(7)	S → ϵ	S.x = ∅
(8)	S' → I	S'.x = 1
(9)	S' → ϵ	S'.x = ∅

Se cambia

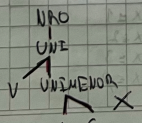
$$UNI \rightarrow IR \mid UNI.x = R.x - 1$$

UNIMENOR → IZ	UNIMENOR.x = 1+Z.x
UNIMENOR → ϵ	UNIMENOR.x = 0

$$UNI \rightarrow IS \mid \epsilon \mid IR \mid VUNIMENOR$$

(4) $UNI \rightarrow IW \mid VUNIMENOR \mid \epsilon \quad P(UNI) = \{I\} \cup \{V\} \cup \{\epsilon\}$

(5) $W \rightarrow SIR \quad P(W) = \{I, \epsilon\} \cup \{V, X\}$



UNI → IW	UNI.x = 1+W.x
UNI → VUNIMENOR	UNI.x = 5+UNIMENOR.x
UNI → ϵ	UNI.x = 0
W → S	W.x = S.x
W → R	W.x = R.x

$P(NO) = \{I, \epsilon\} \cup \{X, \epsilon\}$
$P(DEC) = \{X\} \cup \{\epsilon\} = \{X, \epsilon\}$
$P(UNI) = \{I\} \cup \{V\} \cup \{I, \epsilon\}$
$P(UNIMENOR) = \{I\} \cup \{\epsilon\} = \{I, \epsilon\}$
$P(R) = \{V\} \cup \{X\} = \{V, X\}$
$P(S) = \{I\} \cup \{\epsilon\} = \{I, \epsilon\}$
$P(S') = \{I\} \cup \{\epsilon\} = \{I, \epsilon\}$

$$UNI \rightarrow UNIMENOR \mid VUNIMENOR$$
$$UNIMENOR \rightarrow IR \mid IS \mid \epsilon$$

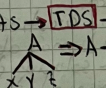
$$\begin{cases} UNIMENOR \rightarrow IZ \mid \epsilon \\ Z.x \rightarrow R.x \\ Z.x \rightarrow S.x \end{cases}$$

```
UNI(){
  if(input=='I'){
      match('I');
      return(1+W());
  }else if(input=='V'){
      match('V');
      return(5+UNIMENOR());
  }else
      return(0);
  }
}
```

```
W(){
  if(input=='V'||input=='X'){
      return(R());
  }else
      return(S());
  }
```

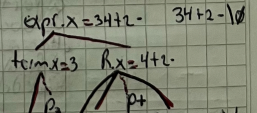
expr → expr + term | expr - term | term
term → 0|1|...|9

3+5 → TDS → 3 5 +



① DIVIDE

BNF	(R.S) Acciones Semánticas
expr → expr + term	expr.x = expr.x || term.x || '+'
expr → expr - term	expr.x = expr.x || term.x || '-'
expr → term	expr.x = term.x
term → 0	term.x = '0'
⋮	⋮
term → 9	term.x = '9'



expr → term R	expr.x = term.x || R.x
R → + term R	R.x = term.x || '+' || R.x
R → - term R	R.x = term.x || '-' || R.x
R → ϵ	R.x = ''
term → 0	term.x = '0' print '0'
⋮	

expr → term R
R → + term {print '+'} R
R → - term {print '-'} R
R → ϵ
term → 0 {print '0'}

Observaciones

- Empezar de adentro del BNF resuelve mucho.
- El árbol sintáctico nos va gritar qué regla utilizar y dónde.
- Instancias de variables sintácticas:
  - Sintetizada: Solo depende de su hijo, son valores de retorno.
  - Heredada: No solo depende de sus hijos sino también de sus hermanos, a través de su padre llega a su hermano. Paso de valor a sus hijos.
- La variable es semántica si es sintetizada.
- El BNF decide el orden de las reglas semánticas. Hay que respetar el orden porque todo vaya bien.
- El vacío 'ε' siempre es el inicializador, va haber casos que no inicializa pero es muy raro.
- Cuando hay una cuestión de vacío 'ε' siempre eliminar al que está más arriba, el demás abajo dejar nomás.
- Atender que la solución del factor común indirecto no dé basura.
- Si está en el mismo lado derecho va crear problema.
- El vacío 'ε' siempre al dise, no hay que preocuparse.
- Reglas Semánticas → Acciones Semánticas: Difícil
- Acciones Semánticas → Reglas Semánticas: Fácil

Definiciones

Sintetizados: Su valor se calcula en un nodo del árbol sintáctico a partir de los valores de sus hijos. La información fluye de abajo hacia arriba (Bottom-Up).
- Hacia arriba (Atributo Sintetizado R.s): Una vez al contado el final de la cadena (cuando R → ε), el resultado acumulado final debe "subir" sintetizando se hasta la raíz (E.val).

Heredados: Su valor se calcula en un nodo del árbol sintáctico a partir de los valores de su padre, sus hermanos o el mismo nodo. La información fluye de arriba hacia abajo o transversalmente.
- Hacia abajo (Atributo Heredado R.h): Necesitas "heredar" o pasar el valor acumulado hasta el momento hacia los nodos inferiores de R para ir operando término a término.

Acciones Semánticas: Es una regla semántica que en vez de estar separado del BNF, se mete dentro del mismo.
- Son fragmentos o bloques de código ejecutable (normalmente escrito entre llaves {...}) que se incrustan directamente dentro del cuerpo de las producciones de una gramática libre de contexto (BNF).